Parameter Estimation of a Discrete-Time System Using PA, OPA, and RLS Algorithms

Student: Ali Ramezanzadeh
Student ID: 40023131

Date: July 7, 2025

1 Introduction

This report presents the parameter estimation of a discrete-time linear system using three estimation algorithms: the Projection Algorithm (PA), the Optimal Projection Algorithm (OPA), and the Recursive Least Squares (RLS). A persistently excited white noise input signal was used to ensure convergence of the estimates. The main objective is to demonstrate and compare the performance of these adaptive estimation methods.

2 System Model

Consider the following discrete-time state-space model:

$$\begin{bmatrix} x_1(t+1) \\ x_2(t+1) \\ x_3(t+1) \end{bmatrix} = \begin{bmatrix} -a_1 & 1 & 0 \\ -a_2 & 0 & 1 \\ -a_3 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1(t) \\ x_2(t) \\ x_3(t) \end{bmatrix} + \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} u(t)$$

$$y(t) = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1(t) \\ x_2(t) \\ x_3(t) \end{bmatrix}$$

where the true parameters are unknown and are given by:

$$a_1 = 0, \quad a_2 = -0.01, \quad a_3 = 0, \quad b_1 = 1, \quad b_2 = 0, \quad b_3 = 2.$$

3 Regressor Form

By expanding the state equations and expressing the output as a function of past outputs and inputs, the system can be written in the regressor form:

$$y(t) = \phi(t-1)^\top \theta_{\text{true}}$$

where the regressor vector is defined as:

$$\phi(t-1) = \begin{bmatrix} y(t-1) \\ y(t-2) \\ y(t-3) \\ u(t-3) \\ u(t-2) \\ u(t-1) \end{bmatrix},$$

and the true parameter vector is:

$$\theta_{\text{true}} = \begin{bmatrix} 0 \\ 0.01 \\ 0 \\ 2 \\ 0 \\ 1 \end{bmatrix}.$$

This representation forms the basis for applying the estimation algorithms.

4 Part (i) - Parameter Estimation

In this part, the following algorithms were implemented to estimate the parameter vector:

4.1 Projection Algorithm (PA)

The update law is:

$$\hat{\theta}(t) = \hat{\theta}(t-1) + \frac{\phi(t-1)}{\|\phi(t-1)\|^2}\, e(t), \quad e(t) = y(t) - \phi(t-1)^\top \hat{\theta}(t-1).$$

4.2 Optimal Projection Algorithm (OPA)

The covariance matrix update:

$$P(t-1) = P(t-2) - \frac{P(t-2)\,\phi(t-1)\,\phi(t-1)^\top\, P(t-2)}{\phi(t-1)^\top\, P(t-2)\,\phi(t-1)},$$

and the parameter update:

$$\hat{\theta}(t) = \hat{\theta}(t-1) + \frac{P(t-2)\,\phi(t-1)}{\phi(t-1)^\top\, P(t-2)\,\phi(t-1)}\, e(t).$$

4.3 Recursive Least Squares (RLS)

The update equations are:

$$K(t) = \frac{P(t-2)\,\phi(t-1)}{1 + \phi(t-1)^\top\, P(t-2)\,\phi(t-1)},$$

$$P(t-1) = P(t-2) - K(t)\,\phi(t-1)^\top\, P(t-2),$$

$$\hat{\theta}(t) = \hat{\theta}(t-1) + K(t)\, e(t).$$

4.4 Part (i) - Results and Analysis

In this section, we present and analyze the results obtained from three standard RLS-based identification methods:

- **RLS (Recursive Least Squares):** Classical formulation without projection.
- **PA (Projection Algorithm):** Incorporates parameter projection to maintain parameter bounds.
- **OPA (Orthogonal Projection Algorithm):** Enhances adaptation by orthogonal projection.

Each method is evaluated based on:

1. The convergence behavior of estimated parameters.
2. The accuracy of the estimated output $\hat{y}(t)$ compared to the true output $y(t)$.
3. The estimation error $e(t) = y(t) - \hat{y}(t)$.

4.4.1 RLS – Recursive Least Squares

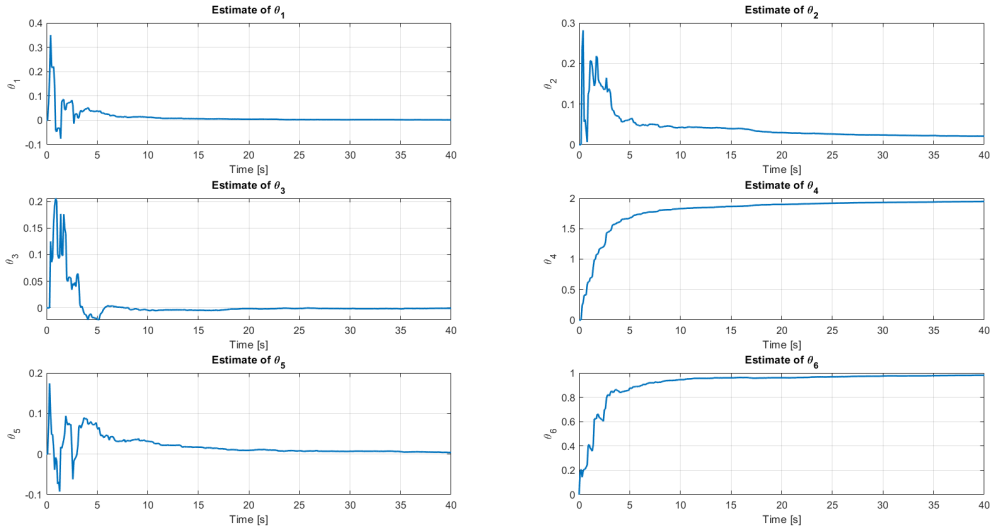


Figure 1: Estimated parameters using classical RLS

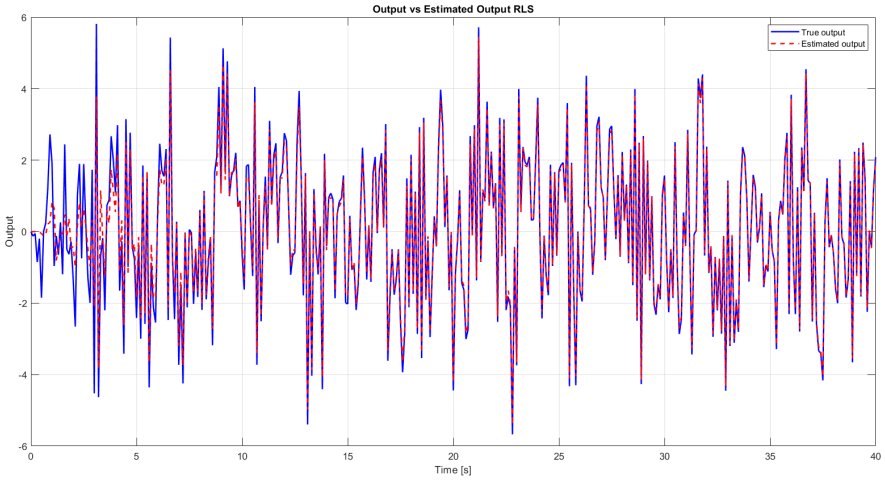


Figure 2: True output vs Estimated output (RLS)

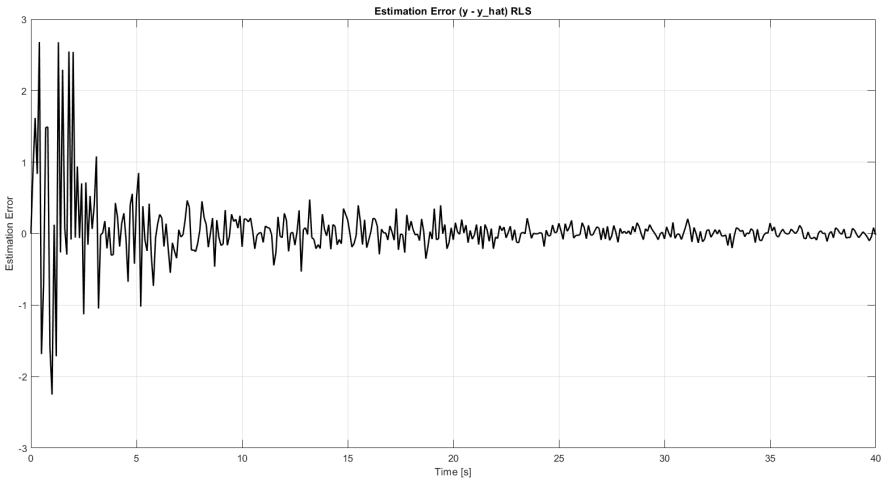


Figure 3: Estimation error $e(t)$ using RLS

Analysis: Among the three methods, classical RLS showed the slowest convergence rate. Although the input was persistently exciting, the lack of projection or additional weighting delayed parameter adaptation. Over time, however, the parameters converged successfully, and the output error decreased steadily.

4.5 PA – Projection Algorithm

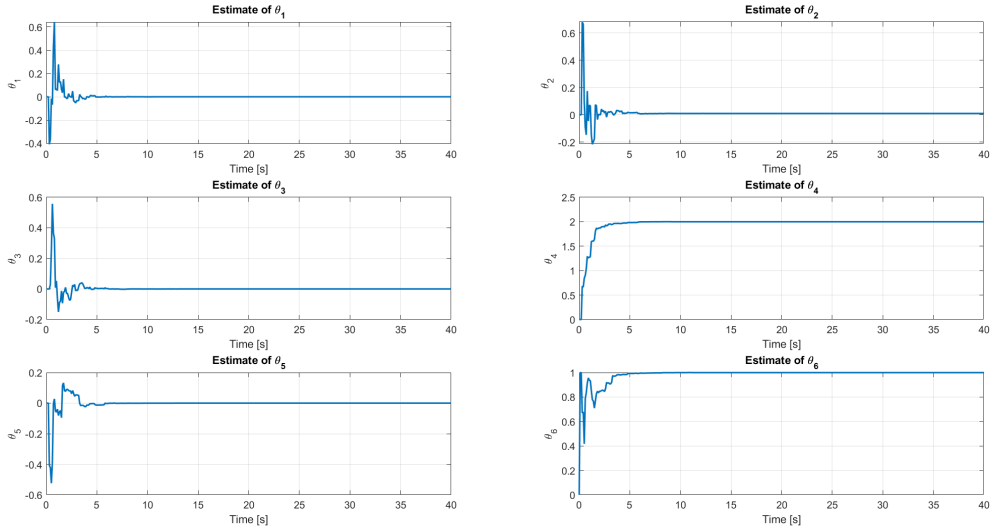


Figure 4: Estimated parameters using PA method

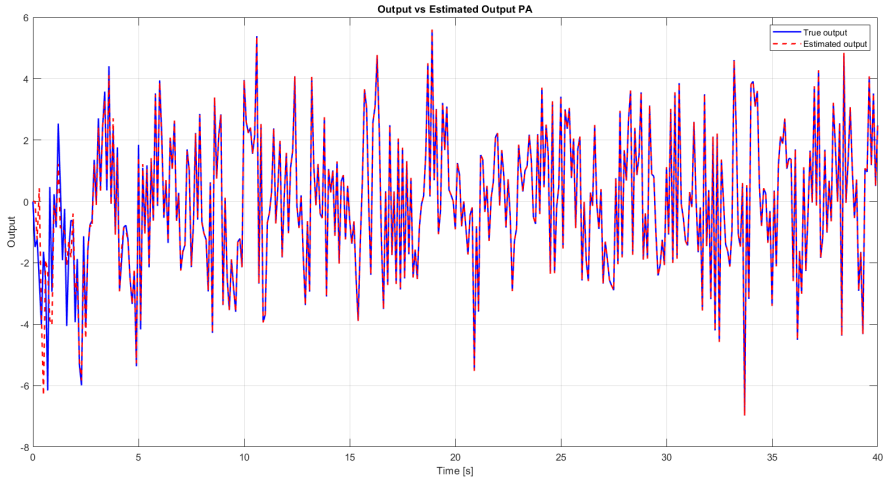


Figure 5: True output vs Estimated output (PA)

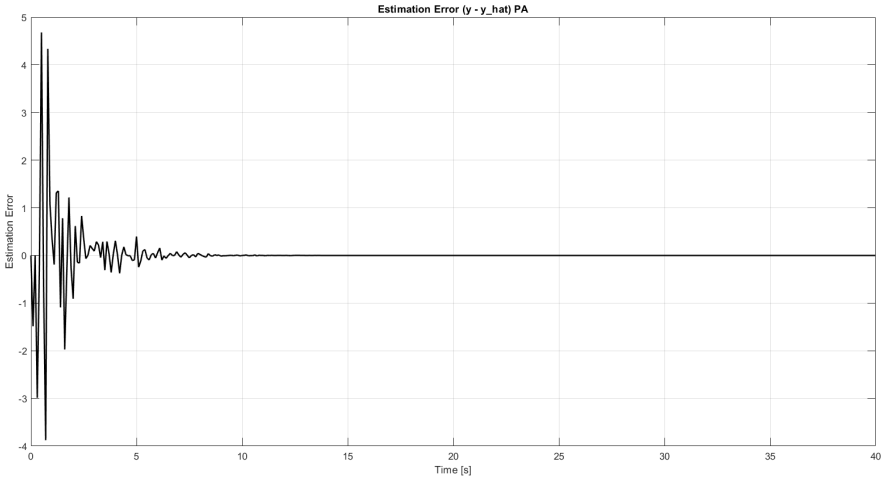


Figure 6: Estimation error $e(t)$ using PA method

Analysis: The Projection Algorithm achieved faster convergence compared to classical RLS. The use of parameter projection ensured that the estimates remained within a feasible region, improving both stability and adaptation speed. The error decreased more rapidly, and tracking performance was more accurate during the transient phase.

4.5.1 OPA – Orthogonal Projection Algorithm

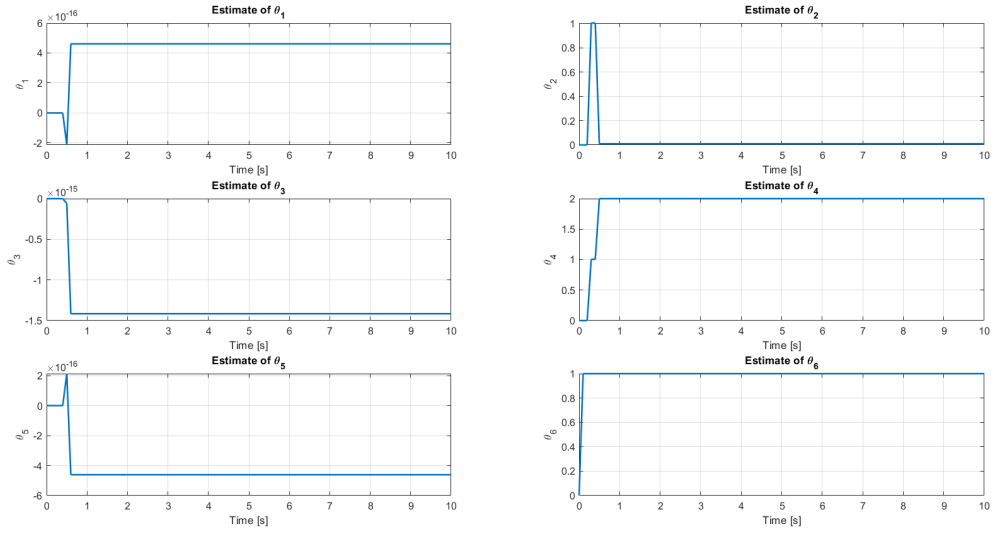


Figure 7: Estimated parameters using OPA method

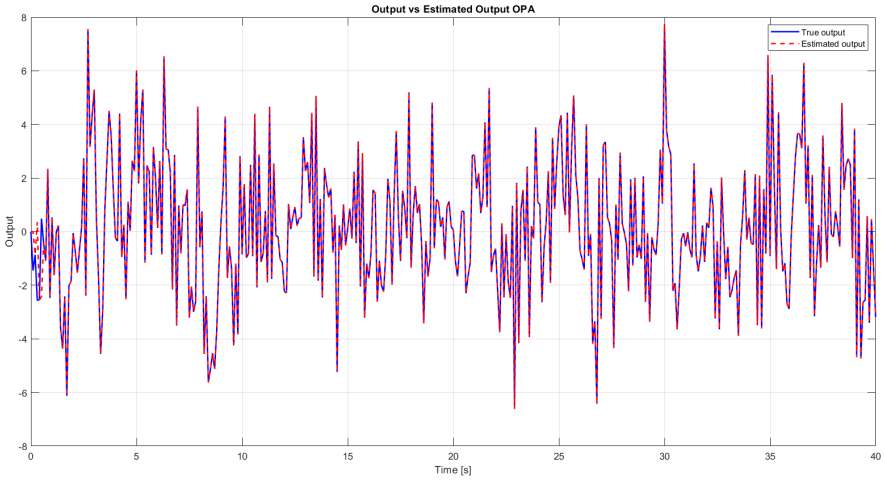


Figure 8: True output vs Estimated output (OPA)

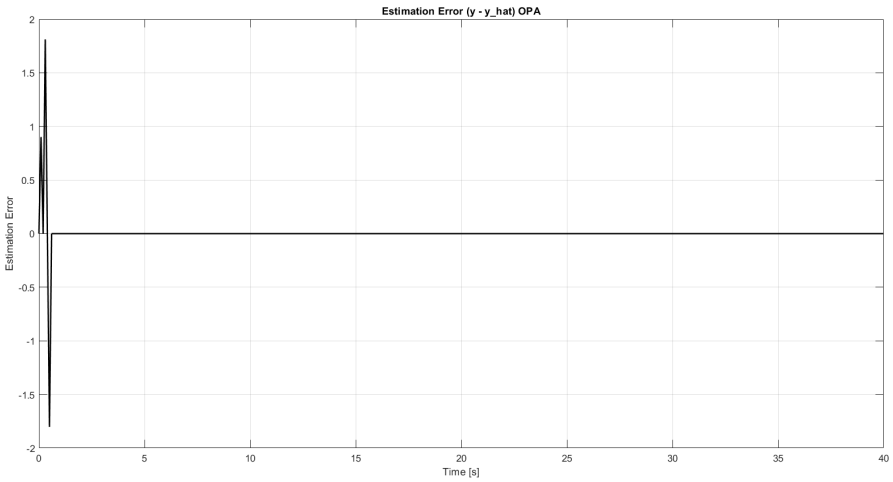


Figure 9: Estimation error $e(t)$ using OPA method

Analysis: Among all three approaches, the Orthogonal Projection Algorithm exhibited the fastest convergence of estimated parameters and the lowest estimation error. The orthogonal projection mechanism allowed for quick adaptation even in the presence of noise, and the tracking of the output was nearly perfect after a short transient period.

5 Part (ii) - Enhanced RLS Algorithms

5.1 Selective Data Weighting RLS

In this method, the update gain is scaled by a factor $a(t-1)$ depending on the signal energy at each time step. The parameter update equations are:

$$
\begin{aligned}
&\text{Compute the prediction error:} && e(t) = y(t) - \phi(t-1)^\top \hat{\theta}(t-1),\\
&\text{Compute energy:} && \nu(t-1) = \phi(t-1)^\top P(t-2)\,\phi(t-1),\\
&\text{Set weighting factor:} && a(t-1) = \begin{cases} a_{\text{high}}, & \text{if } \nu(t-1) \geq \delta,\\ a_{\text{low}}, & \text{otherwise,} \end{cases}\\
&\text{Gain:} && K(t) = \frac{a(t-1)\,P(t-2)\,\phi(t-1)}{1 + a(t-1)\,\phi(t-1)^\top P(t-2)\,\phi(t-1)},\\
&\text{Parameter update:} && \hat{\theta}(t) = \hat{\theta}(t-1) + K(t)\,e(t),\\
&\text{Covariance update:} && P(t-1) = P(t-2) - K(t)\,\phi(t-1)^\top\,P(t-2).
\end{aligned}
$$

Where: - a_{high} is a large weight (e.g., 40), - a_{low} is a small weight (e.g., 0.1), - δ is a threshold for switching between them.

5.2 Exponential Data Weighting RLS

This method discounts older data recursively by updating a time-varying forgetting factor $\alpha(t)$. Unlike the classical approach with a constant λ, here $\alpha(t)$ is computed dynamically. The update equations are:

$$
\begin{aligned}
&\text{Prediction error:} && e(t) = y(t) - \phi(t-1)^\top \hat{\theta}(t-1),\\
&\text{Update forgetting factor:} && \alpha(t) = \alpha_0\,\alpha(t-1) + (1-\alpha_0),\\
&\text{Denominator:} && d(t-1) = \alpha(t-1) + \phi(t-1)^\top\,P(t-2)\,\phi(t-1),\\
&\text{Gain:} && K(t) = \frac{P(t-2)\,\phi(t-1)}{d(t-1)},\\
&\text{Parameter update:} && \hat{\theta}(t) = \hat{\theta}(t-1) + K(t)\,e(t),\\
&\text{Covariance update:} && P(t-1) = \frac{1}{\alpha(t-1)}\Big[P(t-2) - K(t)\,\phi(t-1)^\top\,P(t-2)\Big].
\end{aligned}
$$

5.3 Covariance Resetting RLS

In this approach, the standard Recursive Least Squares (RLS) algorithm is modified to periodically reset the covariance matrix in order to improve numerical stability and prevent divergence. This is done every N steps by reinitializing the covariance matrix P to its initial value.

The update equations are as follows:

Prediction error: $e(t) = y(t) - \phi(t-1)^\top \hat{\theta}(t-1)$,

Denominator: $d(t) = 1 + \phi(t-1)^\top P(t-2)\,\phi(t-1)$,

Gain vector: $K(t) = \dfrac{P(t-2)\,\phi(t-1)}{d(t)}$,

Parameter update: $\hat{\theta}(t) = \hat{\theta}(t-1) + K(t)\,e(t)$,

Covariance update (before reset): $\tilde{P}(t-1) = P(t-2) - K(t)\,\phi(t-1)^\top P(t-2)$,

Covariance resetting rule: $P(t-1) = \begin{cases} \dfrac{1}{k} I, & \text{if } \mathrm{mod}(t, N) = 0, \\ \tilde{P}(t-1), & \text{otherwise,} \end{cases}$

where:

- $\phi(t-1)$ is the regressor vector at time $t-1$,
- $\hat{\theta}(t)$ is the estimated parameter vector at time t,
- $P(t-1)$ is the updated covariance matrix,
- k is the initialization scaling factor for the identity matrix,
- N is the reset period,
- I is the identity matrix.

Remarks:

- This method is useful in cases where model drift or numerical instability may cause standard RLS to degrade over time.
- Resetting the covariance matrix forces the estimator to ”forget” older data periodically, making it more adaptive.
- The choice of N and k affects the trade-off between stability and responsiveness.

Part (ii) – Results and Analysis

This section presents the results of advanced RLS-based estimation methods:

- **Selective Data Weighting RLS:** Adjusts gain dynamically based on regressor energy.
- **Exponential Data Weighting RLS:** Applies an exponentially decaying forgetting factor.
- **Covariance Resetting RLS:** Periodically resets the covariance matrix to improve numerical stability.

Each method is evaluated in terms of:

1. Convergence of the estimated parameters.
2. Tracking of the true output $y(t)$.
3. Estimation error $e(t) = y(t) - \hat{y}(t)$.

Note: All methods were tested under persistently exciting (white noise) input conditions. Although covariance resetting is often preferred for time-varying parameters, in this simulation all parameters were constant.

5.3.1 Selective Data Weighting RLS

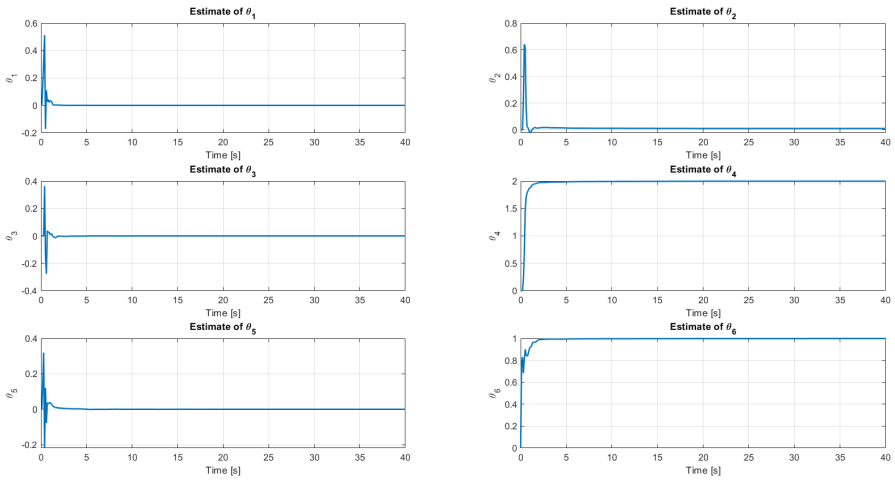


Figure 10: Estimated parameters (Selective Data Weighting RLS)

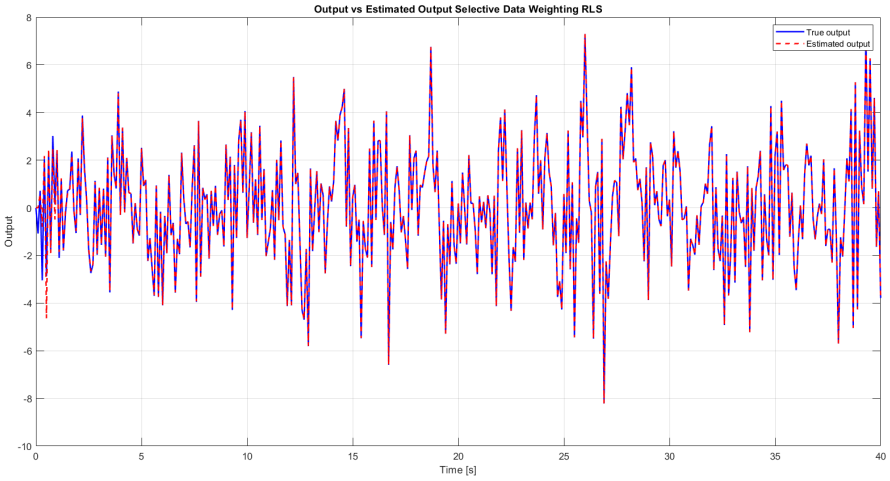


Figure 11: True output vs Estimated output (Selective Data Weighting)

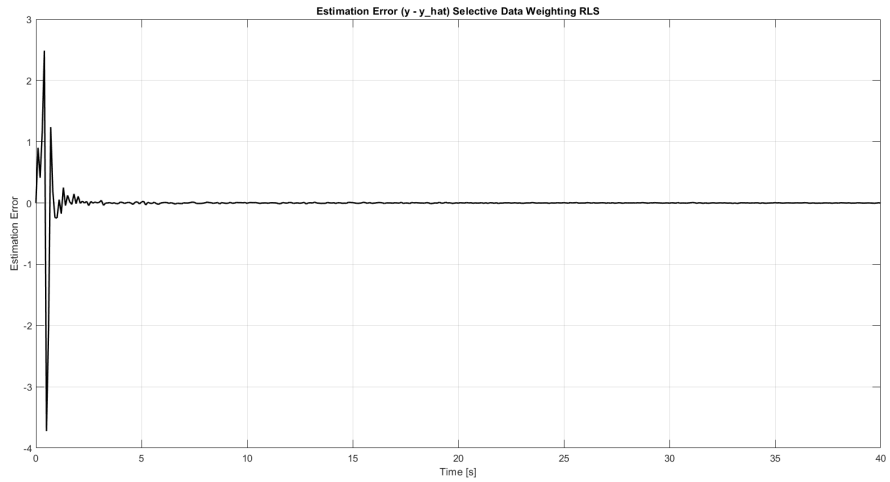


Figure 12: Estimation error $e(t)$ (Selective Data Weighting)

Analysis:
The Selective Data Weighting approach showed moderately fast convergence and stable tracking performance. By increasing the weight when the regressor energy exceeded the threshold, the algorithm improved adaptation during informative data segments. However, compared to Exponential Data Weighting, convergence speed was slower.

5.3.2 Exponential Data Weighting RLS

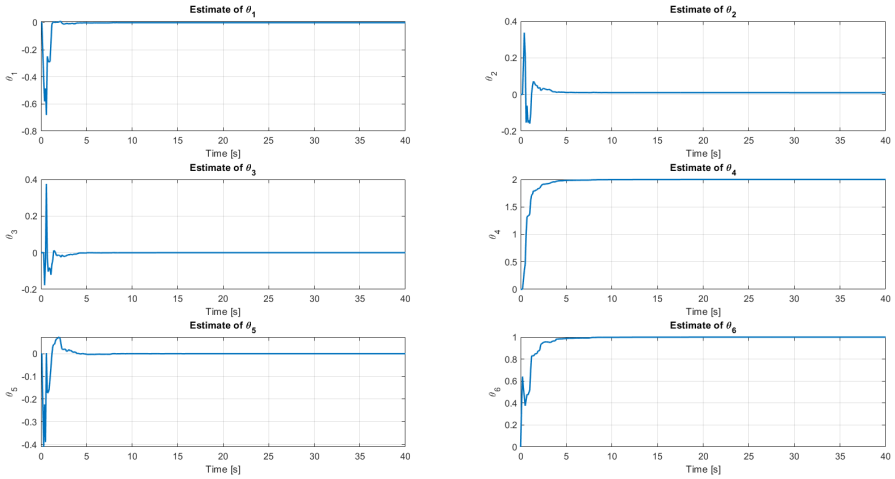


Figure 13: Estimated parameters (Exponential Data Weighting RLS)

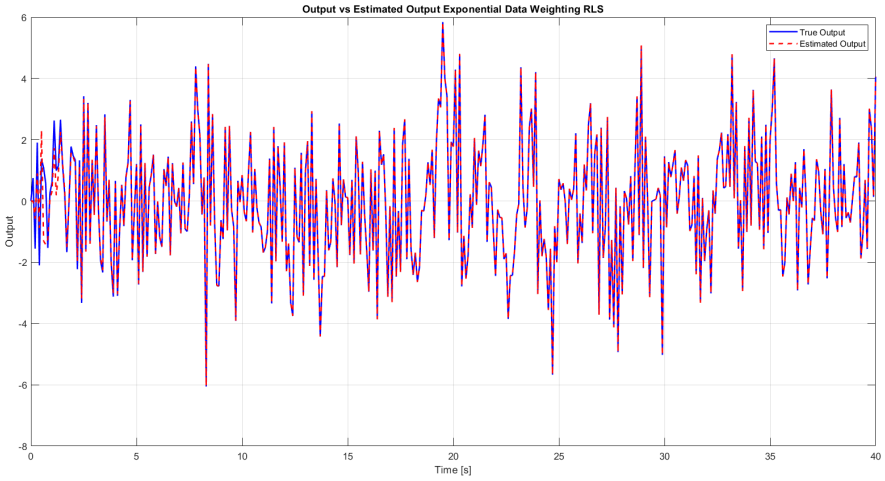


Figure 14: True output vs Estimated output (Exponential Data Weighting)

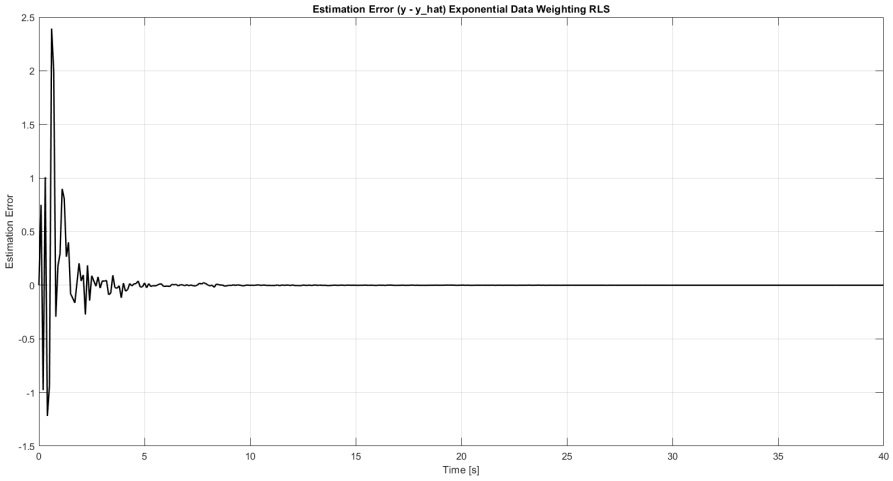


Figure 15: Estimation error $e(t)$ (Exponential Data Weighting)

Analysis:
This method demonstrated the fastest convergence among all enhanced RLS approaches tested. The dynamic forgetting factor helped the estimator quickly adapt to the persistently exciting input, resulting in very low estimation error and excellent output tracking throughout the simulation.

5.3.3 Covariance Resetting RLS

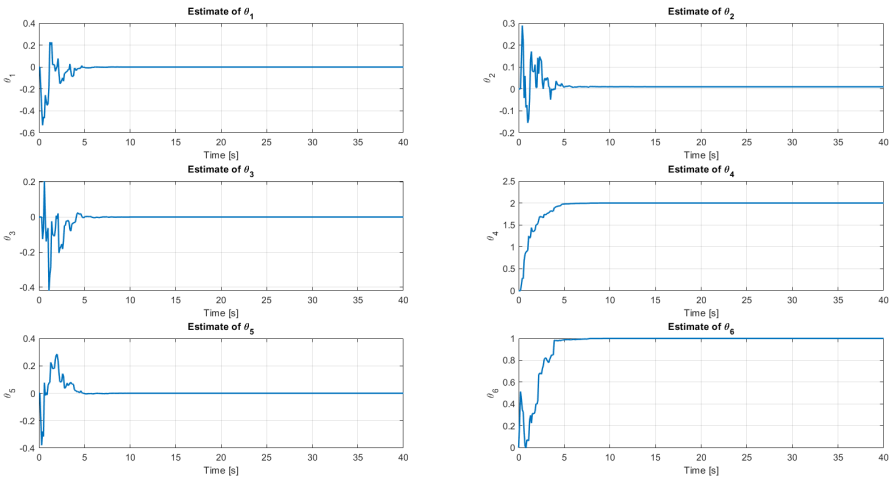


Figure 16: Estimated parameters (Covariance Resetting RLS)

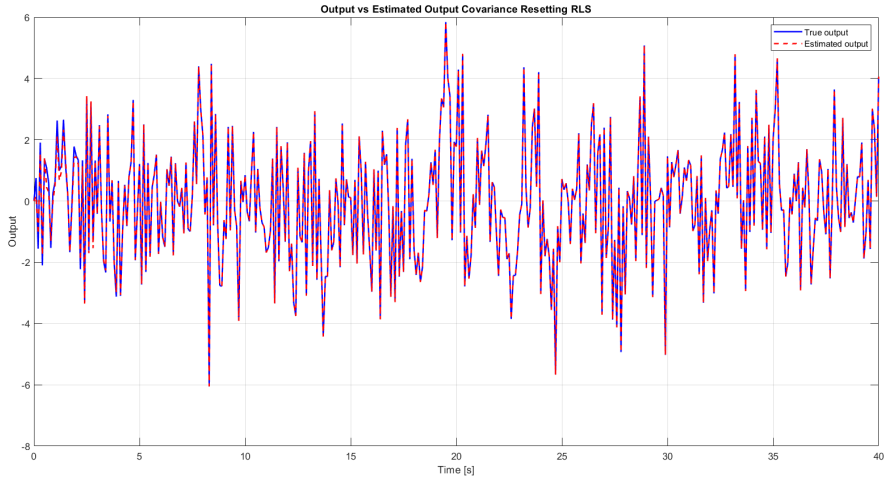


Figure 17: True output vs Estimated output (Covariance Resetting)

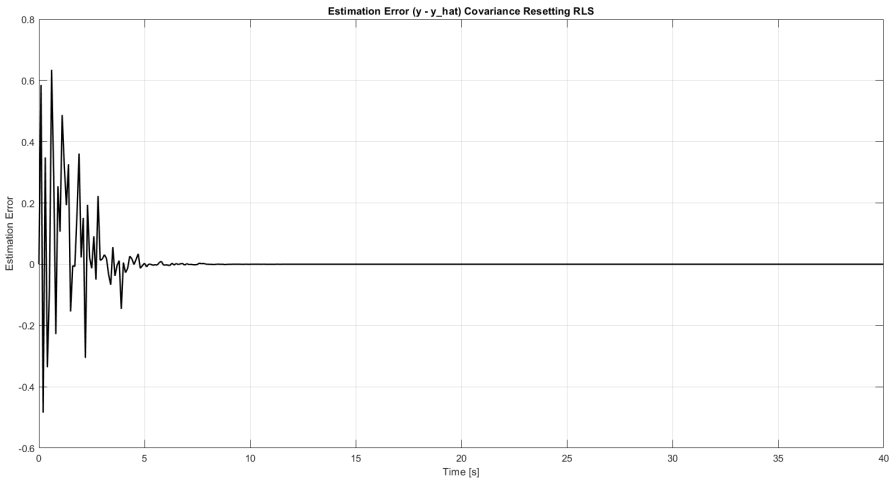


Figure 18: Estimation error $e(t)$ (Covariance Resetting)

Analysis:
Covariance Resetting RLS achieved acceptable convergence speed and maintained estimator stability by periodically reinitializing the covariance matrix. While this method is particularly effective for tracking time-varying parameters, in this constant-parameter scenario it performed slightly slower than Exponential Data Weighting but remained robust throughout the simulation.

6 Part (iii) – Time Varying Parameters

In this part, we consider the identification of a system with parameters that change over time. The true parameter vector $\theta^{\text{true}}(t)$ is piecewise time-varying as follows:

$$\theta^{\text{true}}(t) = \begin{bmatrix} a_1(t) \\ a_2(t) \\ a_3(t) \\ b_3 \\ b_2 \\ b_1 \end{bmatrix}, \quad \text{with} \quad a(t) = \begin{cases} [0, -0.1, 0]^\top, & 0 < t \leq 3, \\ [0.1, 0, -0.2]^\top, & 3 < t \leq 6, \\ [0, 0.1, 0]^\top, & 6 < t \leq 9, \\ [0.1, 0, -0.1]^\top, & t > 9, \end{cases} \quad b = \begin{bmatrix} 2 \\ 0 \\ 1 \end{bmatrix}.$$

6.1 Covariance Modification RLS

To effectively track these time-varying parameters, we apply the Covariance Modification Recursive Least Squares (RLS) algorithm. The key idea is to adapt the covariance matrix by adding a modification term that depends on the prediction error, enabling faster tracking of parameter changes.

The algorithm is described by the following update equations:

Prediction error: $e(t) = y(t) - \phi(t-1)^\top \hat{\theta}(t-1)$,

Gain vector: $K(t) = \dfrac{P(t-1)\,\phi(t-1)}{1 + \phi(t-1)^\top P(t-1)\phi(t-1)}$,

Parameter update: $\hat{\theta}(t) = \hat{\theta}(t-1) + K(t)e(t)$,

Covariance update with modification: $P(t-1) = \Big(I - K(t)\phi(t-1)^\top\Big)P(t-2) + \lambda_Q e(t)^2 I$,

where:

- $\phi(t)$ is the regressor vector composed of past inputs and outputs at time t,
- $P(t)$ is the covariance matrix,
- $\lambda_Q > 0$ is the covariance modification factor tuning the adaptation speed,
- I is the identity matrix.

Remarks:

- The additional term $\lambda_Q e(t)^2 I$ inflates the covariance when the prediction error is large, increasing the estimator's sensitivity to parameter changes.
- This approach allows the estimator to forget old information faster and track abrupt or gradual parameter variations.
- Proper tuning of λ_Q is crucial to balance noise sensitivity and tracking performance.

The simulation results demonstrate that Covariance Modification RLS can successfully track the time-varying parameters, as shown in the plots of estimated parameters versus true parameters and the output estimation error.

Figures:

- Estimated parameter trajectories vs. true time-varying parameters.
- Output signal vs. estimated output.

6.2 Part (iii) – Results

This section evaluates the performance of the different algorithms when estimating parameters that vary over time. Four methods were tested:

- **Standard RLS**
- **Projection Algorithm (PA)**
- **Orthogonal Projection Algorithm (OPA)**
- **Modified RLS with Covariance Adjustment**

Time-varying parameters were imposed to simulate realistic adaptation scenarios. All estimators received the same persistently exciting (white noise) input.

6.2.1 Estimated Parameters over Time

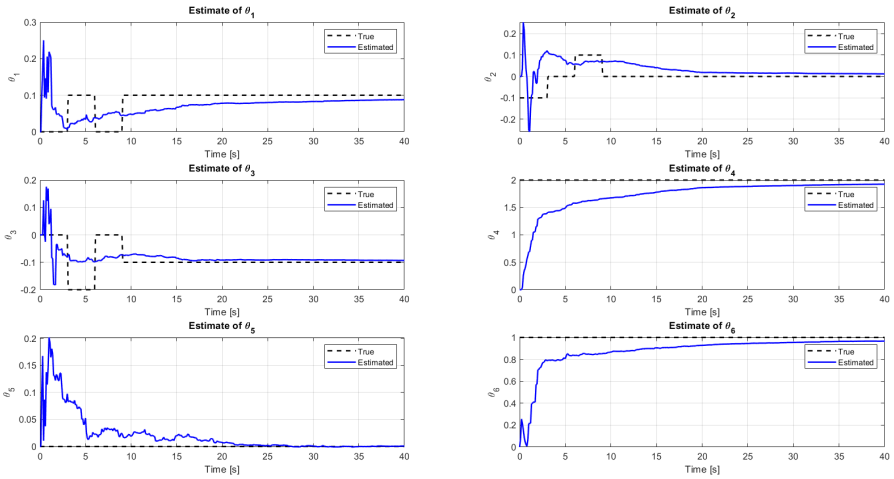


Figure 19: Estimated parameters – Standard RLS

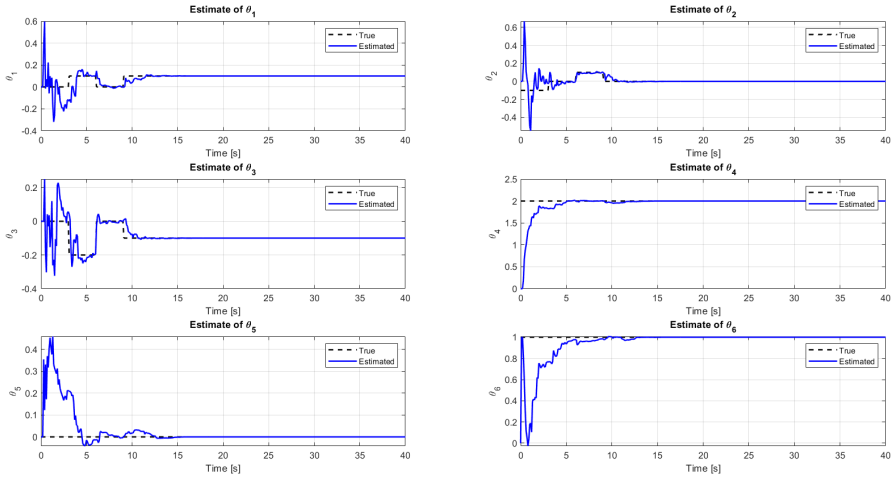


Figure 20: Estimated parameters – Projection Algorithm (PA)

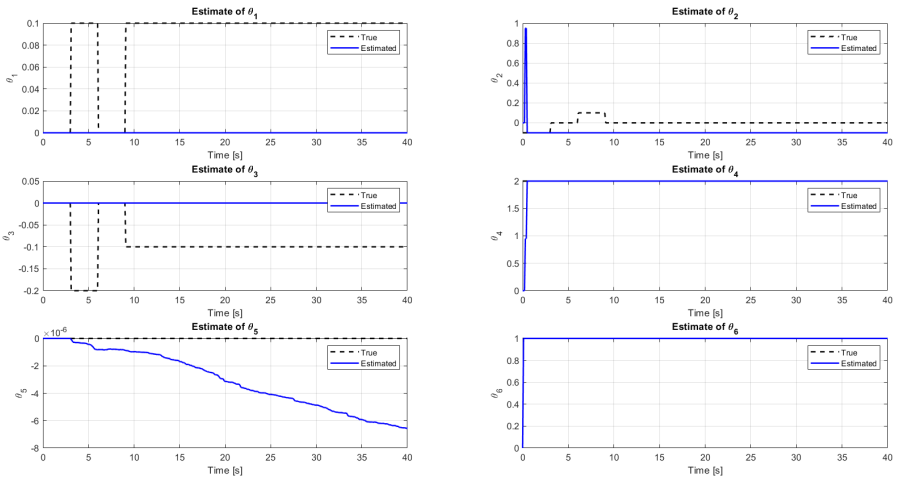


Figure 21: Estimated parameters – Orthogonal Projection Algorithm (OPA)

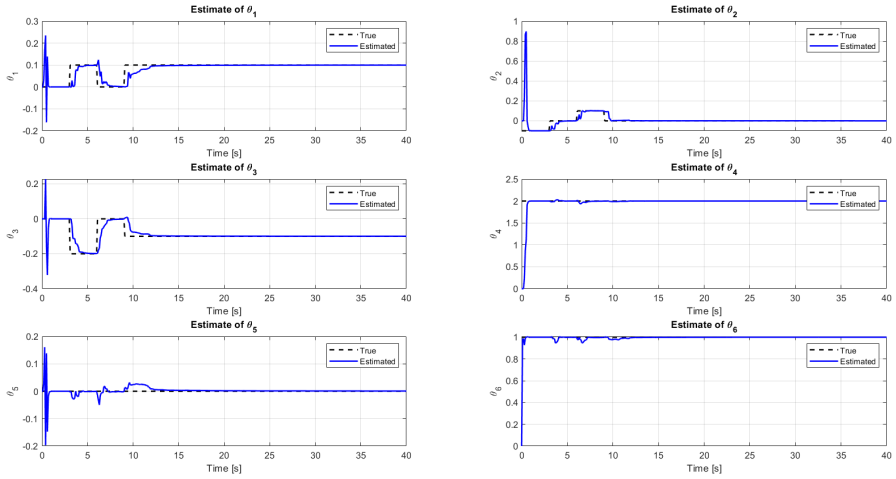


Figure 22: Estimated parameters – Modified RLS

6.2.2 Estimation Errors Comparison

To directly compare estimation performance, the error signals $e(t) = y(t) - \hat{y}(t)$ of all four methods are shown together:

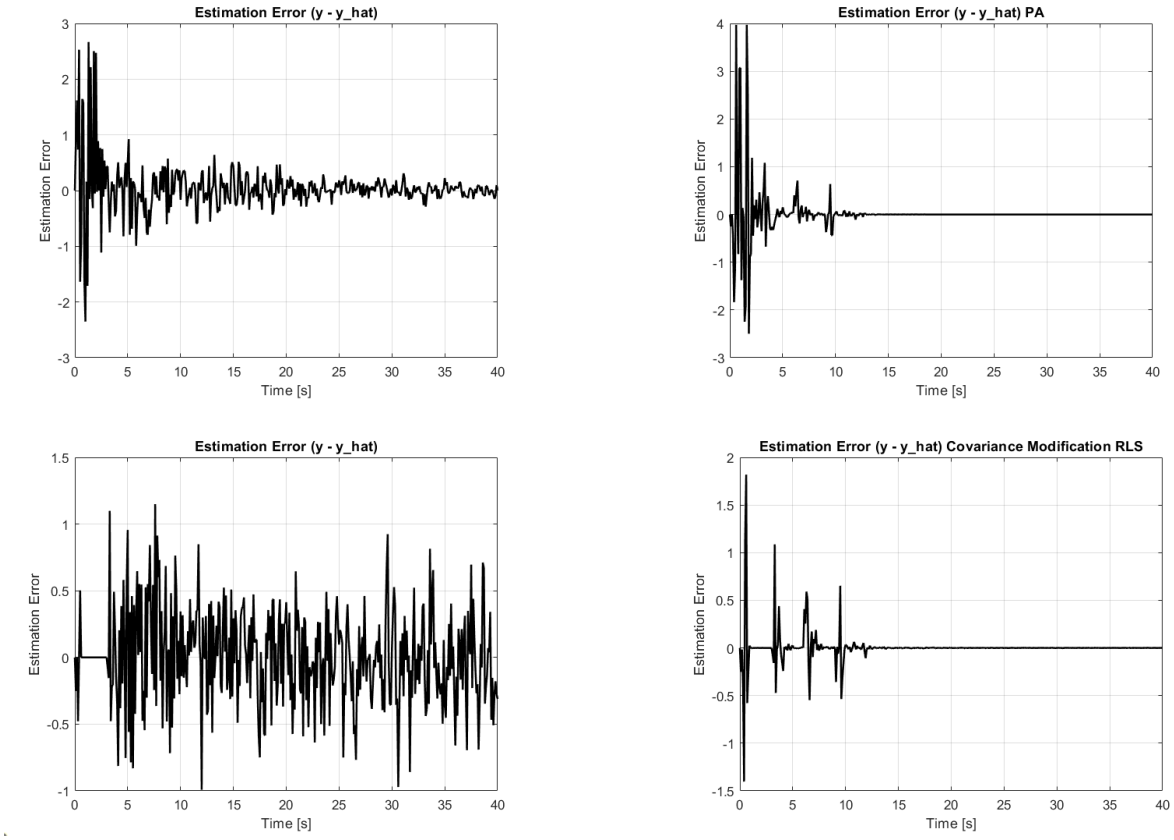


Figure 23: Estimation errors comparison: Top-left RLS, Top-right PA, Bottom-left OPA, Bottom-right Modified RLS

6.2.3 Analysis and Discussion

Modified RLS achieved the best performance in this scenario, showing fast adaptation and low steady-state error when parameters changed. The covariance adjustment mechanism allowed the estimator to maintain sufficient sensitivity to parameter variations.

Projection Algorithm (PA) performed reasonably well, achieving convergence and tracking the parameter variations better than standard RLS. However, its adaptation speed was slightly slower than Modified RLS.

Standard RLS converged adequately but showed slower adaptation to parameter changes due to the lack of explicit forgetting or covariance reinitialization.

Orthogonal Projection Algorithm (OPA) was the least effective in this setting. As time progressed, its covariance matrix decayed too quickly, causing the algorithm to lose sensitivity and fail to follow the time-varying parameters, leading to divergence or very slow response.

Overall Conclusion:

- For scenarios with slowly or rapidly time-varying parameters, **Modified RLS** is the recommended approach.
- PA offers a balance between stability and adaptiveness.
- Standard RLS may suffice when parameters are nearly constant.
- OPA is not suitable in time-varying conditions without modifications.